

LECTURE 8

Git, Docker & PostgreSQL

Modern DevOps and Development Workflow

Today's Agenda

- 1 Git fundamentals and workflows
- 2 Branching strategies for teams
- 3 Docker containerization
- 4 PostgreSQL databases
- 5 Development environment setup
- 6 Deployment best practices

PART 1

Version Control with Git

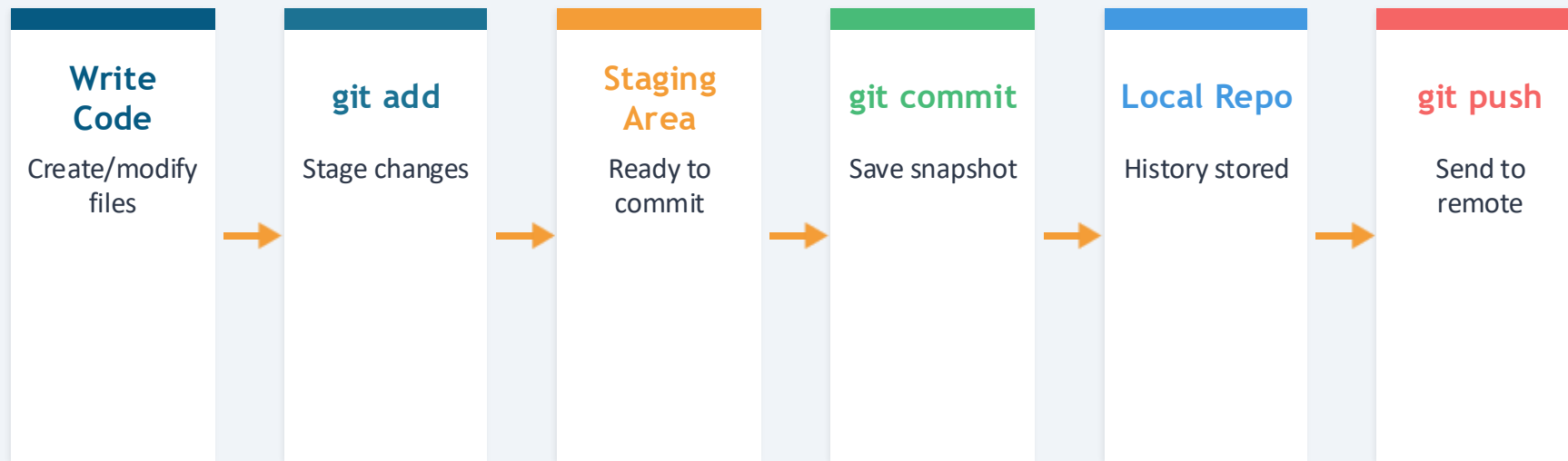
Managing code changes and collaboration

Why Version Control Matters

- Track changes over time
- Collaborate with team members
- Revert to previous versions
- Branch for parallel development
- Merge code safely
- Maintain project history

Git Workflow

From local edits to remote repository



Essential Git Commands

BASH

```
git init           # Initialize repository
git clone <url>    # Copy remote repo
git status         # Check file status
git add <file>     # Stage changes
git commit -m "message" # Create snapshot
git push          # Send to remote
git pull          # Fetch and merge
git log           # View history
```

Git Branching Models

- main/master: Production-ready code
- develop: Integration branch
- feature/xyz: New feature branches
- bugfix/xyz: Bug fix branches
- release/x.x: Release preparation
- Prevents conflicts and enables parallel work

Working with Branches

BASH

```
# Create a new feature branch
git checkout -b feature/user-login

# Make changes, commit
git add .
git commit -m "Implement user login"

# Push branch to remote
git push -u origin feature/user-login

# Create Pull Request for review
# After approval, merge to develop/main
git checkout develop
git merge feature/user-login
```


Pull Request Process

- Create feature branch from develop
- Commit and push changes
- Open Pull Request (PR) on GitHub
- Request code review from teammates
- Address review feedback
- Merge after approval
- Delete feature branch

Handling Merge Conflicts

- Conflicts occur when same lines change
- Git marks conflicts with markers
- Manually resolve conflicting sections
- Test thoroughly after resolving
- Commit resolution
- Push to complete merge

PART 2

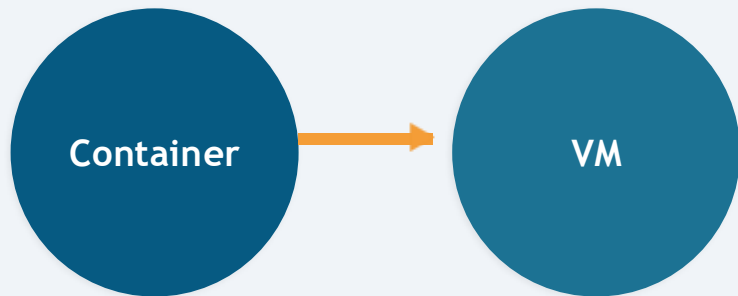
Docker Containerization

Packaging applications for consistency

Introduction to Docker

- Containerization platform
- Packages app + dependencies + runtime
- Lightweight alternative to VMs
- Ensures consistency across environments
- Enables microservices architecture
- Industry standard for deployments

Container vs Virtual Machine



- Containers: Lightweight, shares host kernel
- VMs: Full OS per instance, heavier
- Containers: Seconds to start
- VMs: Minutes to start
- Containers: Better resource utilization
- VMs: More isolation and security

Docker Key Concepts

Image

Template/blueprint for containers

Container

Running instance of an image

Dockerfile

Script to build images

Docker Compose

Multi-container orchestration

Volume

Persistent data storage

Network

Container-to-container communication

Creating a Dockerfile

DOCKERFILE

```
FROM openjdk:11-jre-slim
WORKDIR /app

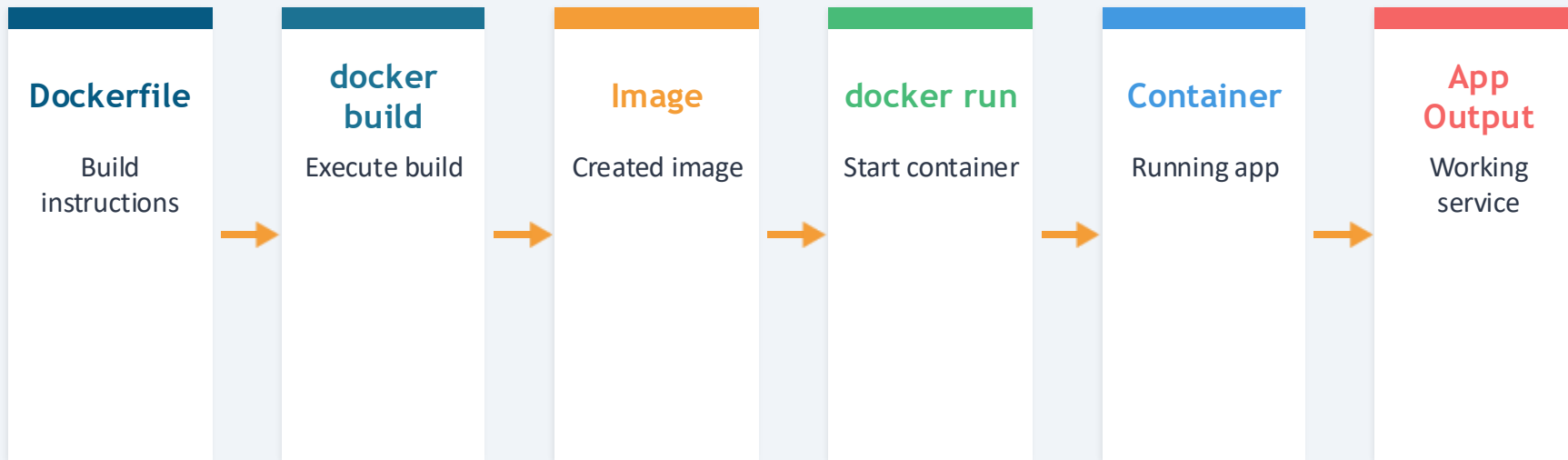
# Copy JAR file
COPY target/myapp.jar app.jar

# Expose port
EXPOSE 8080

# Run application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile defines how to build image

Docker Build Pipeline



Essential Docker Commands

BASH

```
docker build -t myapp:1.0 .      # Build image
docker run -p 8080:8080 myapp:1.0 # Run container
docker ps                        # List running
docker stop <container_id>      # Stop container
docker logs <container_id>      # View logs
docker images                    # List images
docker rm <container_id>        # Remove container
```

Docker Compose for Multiple Services

YAML

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: jdbc:postgresql://db:5432/mydb
    depends_on:
      - db
  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: password
```

Development without Docker vs with Docker

Without Docker

- Works on my machine syndrome
- Different environments between devs
- Complex setup instructions
- Database setup per developer
- Dependency conflicts
- Deployment surprises



With Docker

- Same environment everywhere
- Single docker-compose.yml file
- One-command setup: docker-compose up
- Database runs in container
- Isolated dependencies
- Predictable deployments

PART 3

PostgreSQL Databases

Modern relational database system

PostgreSQL Advantages

- Open-source and free
- ACID compliance for transactions
- Advanced query features (JSON support)
- Powerful indexing
- Security features (roles, permissions)
- Excellent for Spring Boot applications

Running PostgreSQL in Docker

BASH

```
docker run --name postgres \  
  -e POSTGRES_PASSWORD=password \  
  -e POSTGRES_DB=mydb \  
  -p 5432:5432 \  
  -d postgres:13  
  
# Connect with psql  
psql -h localhost -U postgres -d mydb
```

JPA and Hibernate

- JPA: Java Persistence API standard
- Hibernate: Most popular JPA implementation
- ORM: Object-Relational Mapping
- Map Java classes to database tables
- Automatic CRUD operations
- Query Language: HQL or JPQL

JPA Entity Example

JAVA

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true)
    private String email;

    @OneToMany(mappedBy = "user")
    private List<Post> posts;
}
```


Spring Data JPA Repository

JAVA

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    User findByEmail(String email);

    List<User> findByNameContains(String name);

    @Query("SELECT u FROM User u " +
        "WHERE u.age > :age")
    List<User> findAdultUsers(@Param("age") int age);
}
```

PART 4

Discussion: Git & Team Workflow

Collaborative development challenge

Git Workflow Challenge

12 min

SCENARIO

Your team of 5 developers is building a Spring Boot app. Developer A writes a feature on Monday. Developer B modifies the same file on Tuesday. Developer C wants to deploy on Wednesday. Without Git, they email code files back and forth.

THINK ABOUT

1

What problems arise without version control?

2

How do branches prevent conflicts?

3

What's your team's branching strategy?

4

How would you integrate changes safely?

Think — Pair — Share

5 min

Real scenario: legacy system modernization

A startup runs their database directly on a developer's laptop. They decide to dockerize everything. What are the first 3 steps, and what could go wrong during migration?

1 THINK

Reflect individually for 1 minute

2 PAIR

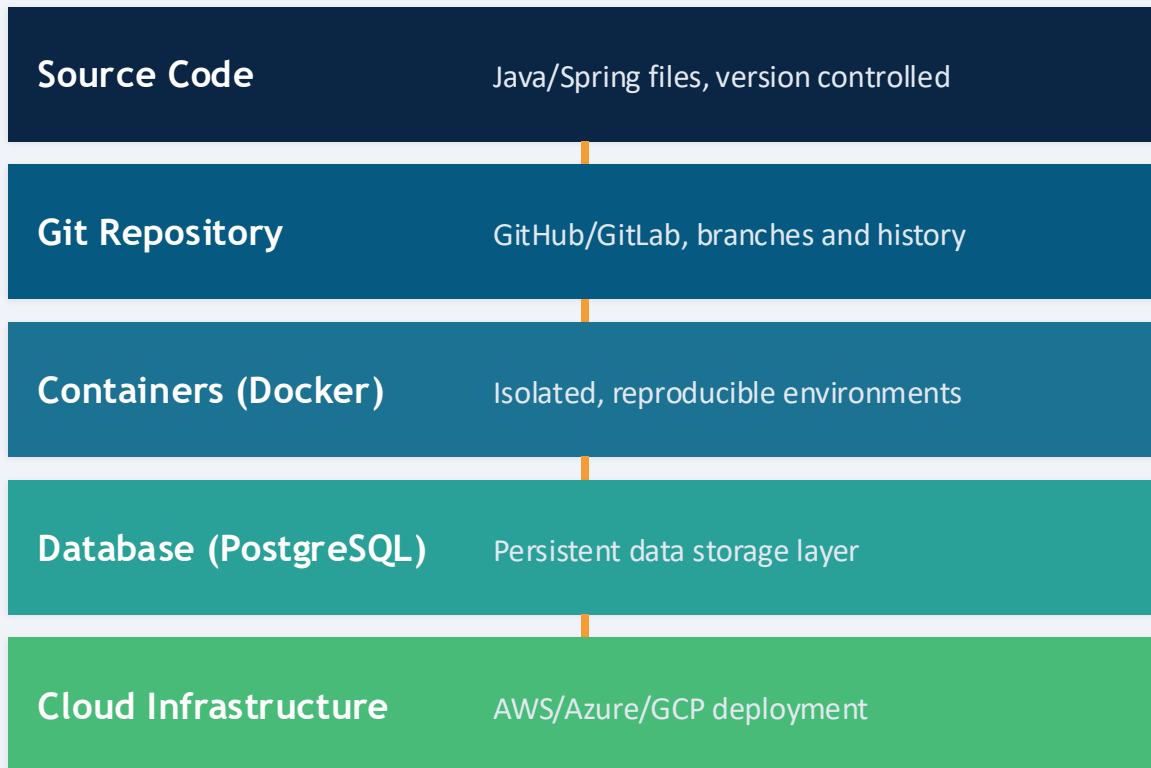
Discuss with your neighbor for 2 minutes

3 SHARE

Share insights with the class

Hints: Data persistence with Docker volumes | Connection strings change | Environment variables | Database backup and migration

Modern Development Stack

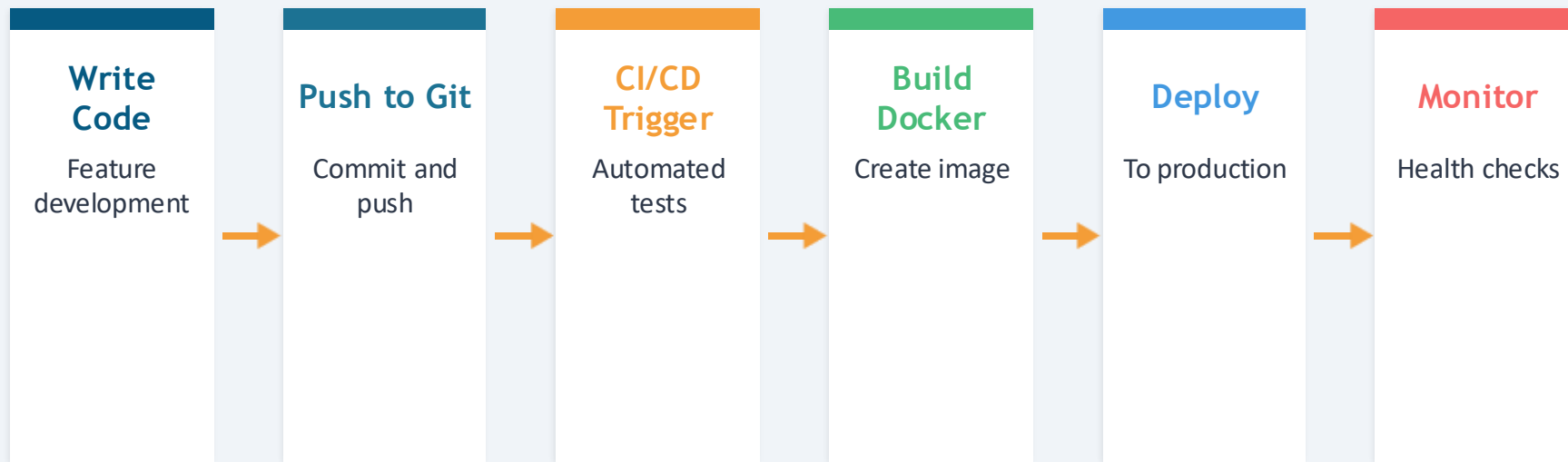


Key Points

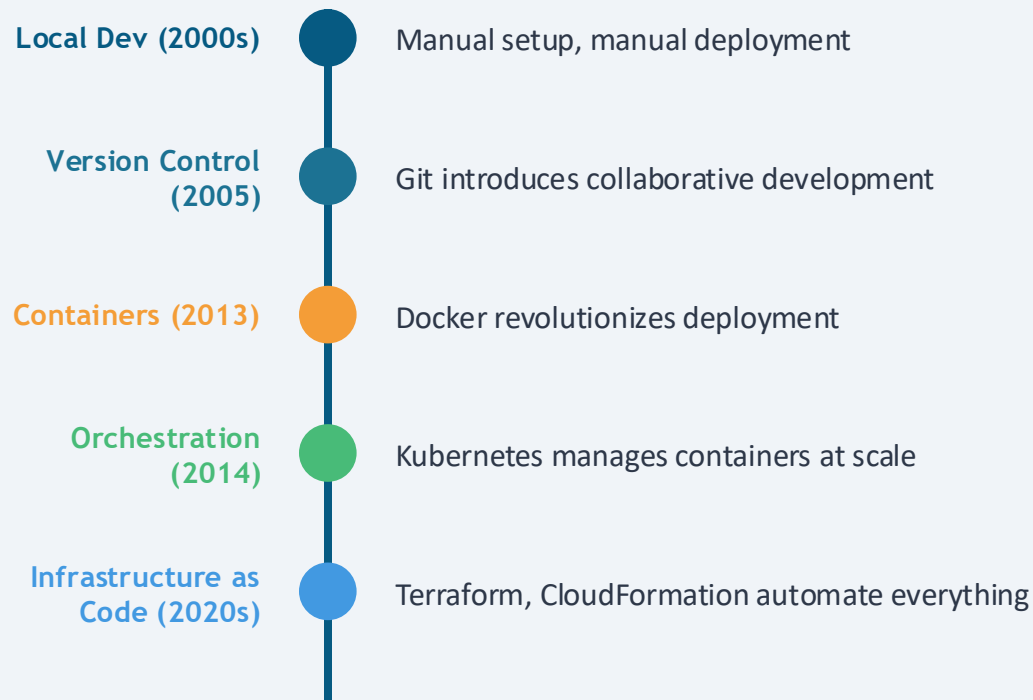
- CI/CD pipelines
- Auto-scaling
- Load balancing
- Monitoring

DevOps Pipeline

From code to production



DevOps Evolution Timeline



Database Schema Migrations

- Liquibase: Track schema changes
- Flyway: Versioned SQL scripts
- Spring Boot integration: automatic migrations
- Version-controlled schema history
- Rollback capability
- Environment-specific scripts

Database Migration with Liquibase

XML

```
<changeSet id="1" author="dev">
  <createTable tableName="users">
    <column name="id" type="BIGINT"
      autoIncrement="true">
      <constraints primaryKey="true"/>
    </column>
    <column name="email" type="VARCHAR(255)">
      <constraints nullable="false" unique="true"/>
    </column>
  </createTable>
</changeSet>
```

Git Best Practices

- Write clear commit messages
- Commit frequently with logical units
- Create descriptive branch names
- Use .gitignore for sensitive/build files
- Review before merging
- Keep main/master clean and deployable
- Use semantic versioning for tags

Commit Message Guidelines

BASH

```
# GOOD: Concise, action-oriented
git commit -m "Add user authentication endpoint"
git commit -m "Fix null pointer in UserService"
git commit -m "Refactor database query for performance"

# BAD: Vague or incomplete
git commit -m "updates"
git commit -m "bug fixes and stuff"
git commit -m "changes"
```

Docker Security Best Practices

- Use official base images
- Minimize layers and size
- Run as non-root user
- Scan images for vulnerabilities
- Use private registries for company code
- Keep Docker and OS updated

Spring Boot PostgreSQL Configuration

PROPERTIES

```
# application.properties
spring.datasource.url=\
    jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

```
# In Docker: use service name
spring.datasource.url=\
    jdbc:postgresql://db:5432/mydb
```

Data Persistence in Docker

- Containers are ephemeral by default
- Volumes persist data beyond container lifecycle
- Named volumes for database data
- Bind mounts for development
- Volume backup and restore strategies
- Avoid storing critical data in containers

Environment-Specific Configuration

PROPERTIES

```
# application-dev.properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=create-drop
```

```
# application-prod.properties
spring.datasource.url=\
    jdbc:postgresql://prod-db:5432/mydb
spring.jpa.hibernate.ddl-auto=validate
```

```
# application.yml (with profiles)
spring:
  profiles: dev
  datasource:
    url: jdbc:h2:mem:testdb
```

Integration Testing with Testcontainers

JAVA

```
@DataJpaTest
@Testcontainers
class UserRepositoryTest {
    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:13");

    @Test
    void testFindByEmail() {
        User user = new User("test@example.com");
        repository.save(user);

        User found = repository.findByEmail(
            "test@example.com"
        );
        assertThat(found).isNotNull();
    }
}
```


Key Takeaways

- 1 Git enables team collaboration and version history
- 2 Branching strategies prevent conflicts
- 3 Docker ensures consistent environments
- 4 PostgreSQL is production-ready database
- 5 Modern stack: Git + Docker + PostgreSQL + Spring Boot
- 6 DevOps: Automate from code to deployment
- 7 Security and monitoring are essential

Questions?

Next: Lecture 9 — Spring Boot Introduction

CSCI 2450 | Computer Interface Programming | 2026

Resources & References

- Git: <https://git-scm.com/>
- Docker: <https://www.docker.com/>
- PostgreSQL: <https://www.postgresql.org/>
- Spring Data JPA: <https://spring.io/projects/spring-data-jpa>
- Liquibase: <https://www.liquibase.org/>
- Testcontainers: <https://www.testcontainers.org/>

Advanced Topics

- Kubernetes for container orchestration
- CI/CD with Jenkins, GitLab CI, GitHub Actions
- Database replication and backup strategies
- Microservices architecture patterns
- API gateways and service discovery
- Monitoring and logging with ELK stack

Practice Exercises

- Create a Git repository and practice branching
- Write a Dockerfile for a Spring Boot app
- Set up PostgreSQL in Docker with Docker Compose
- Create JPA entities for a simple domain
- Implement Spring Data repositories
- Write integration tests with Testcontainers